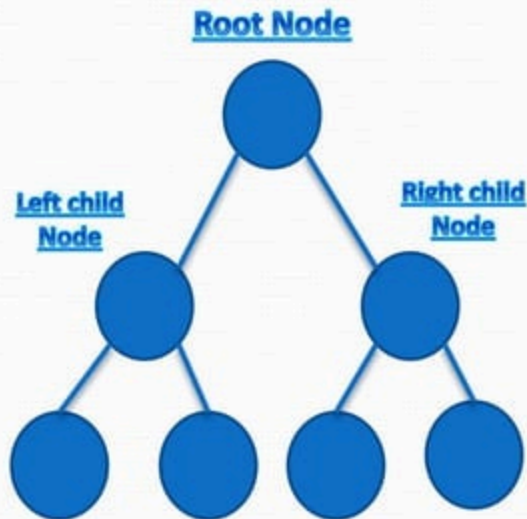


# BASIC CONCEPT

AVL tree is the special case of  
BINARY SEARCH TREE.

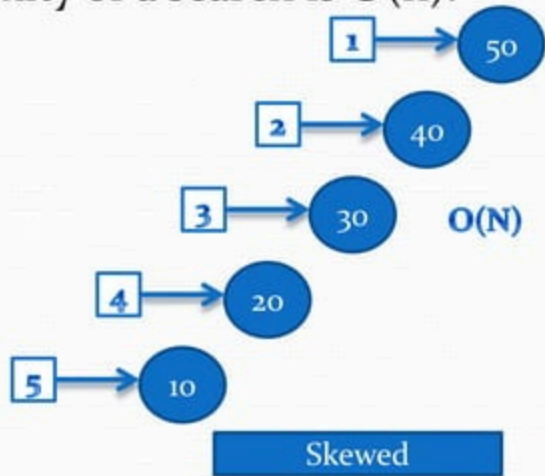
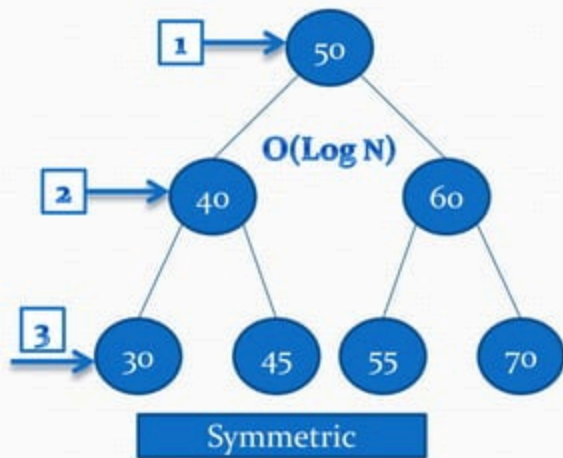
## WHAT IS BINARY SEARCH TREE

- Every node contain only two child.
- Small value at left child node and large value at right child node as compare to Root node.
- The main advantage of BST is that it is easy to perform operations like **inserting, searching, traversing and deleting.**



# PROBLEM WITH BST

- The disadvantage of skewed binary search tree is that the worst case time complexity of a search is  $O(n)$ .



# NEED TO RESOLVE PROBLEM

- There is a need to maintain the binary search tree to be of the balanced height, so that it is possible to obtain for the search operation a time complexity of  **$O(\log N)$**  in the worst case.
- One of the most popular balanced tree was introduced by **Andelson-Velskii and Landis(AVL)**

# DEFINITION OF AVL TREE

A binary tree is said to be an **AVL tree** if **T** is a root of tree and **T(L)** is its left sub tree and **T(R)** is its right sub-tree of tree **T** and **H(T(L))** and **H(T(R))** are the heights of the left and right sub-trees of **T** respectively, and  $|H(T(L)) - H(T(R))| \leq 1$  Then we called **T** is AVL tree.

## Balance factor

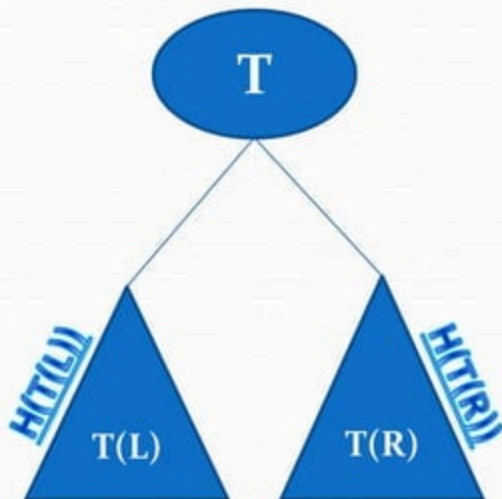
Height of left sub-tree **minus** height of Right sub-tree

$$[H(T(L)) - H(T(R))]$$

Note:

- An empty binary tree is an AVL Tree
- Balance Factor should be

**-1   0   1**



# EXAMPLE

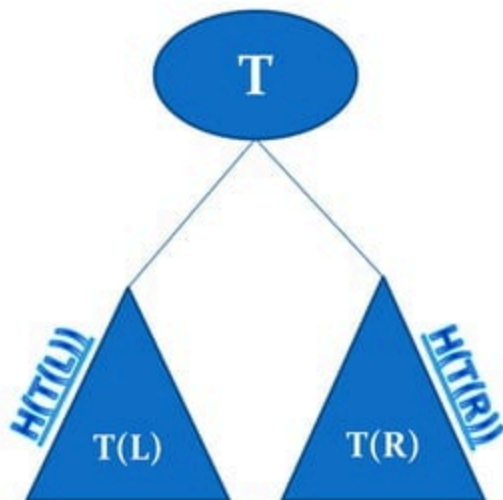
$$\underline{H(T(L)) = 3}$$

$$\underline{H(T(R)) = 2}$$

$$\underline{BF = H(T(L)) - H(T(R))}$$

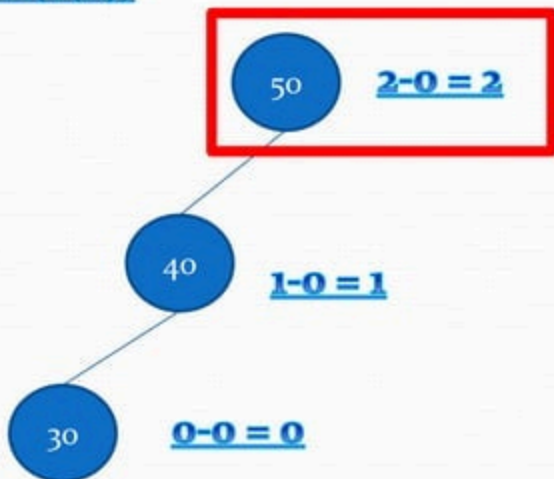
$$\underline{BF = 3 - 2}$$

$$\underline{BF = 1}$$



# EXAMPLE

$$\text{BF} = \text{H}(\text{T}(\text{L})) - \text{H}(\text{T}(\text{R}))$$



**UNBALANCE TREE**

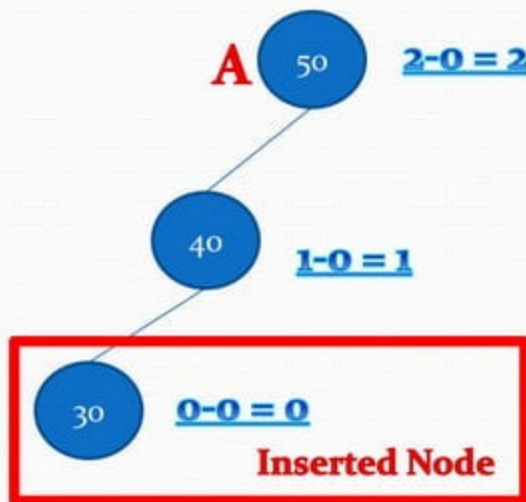
# ROTATION

- If after the insertion of the element the, the balance factor of any node is affect this problem is over come by using **rotation**.
- **Rotation** is use to restore the balance of search tree.



# ROTATION

- To perform the rotation it is necessary to identify a specific node **A** whose balance factor (BF) is neither 0, -1 or 1 and which is the nearest ancestor to the inserted node on the path from the inserted node to the root.





# TYPES OF ROTATION

LL(Left Left) rotation.

RR(Right Right) rotation.

} Single Rotation

RL(Right Left) rotation.

LR( Left Right) rotation

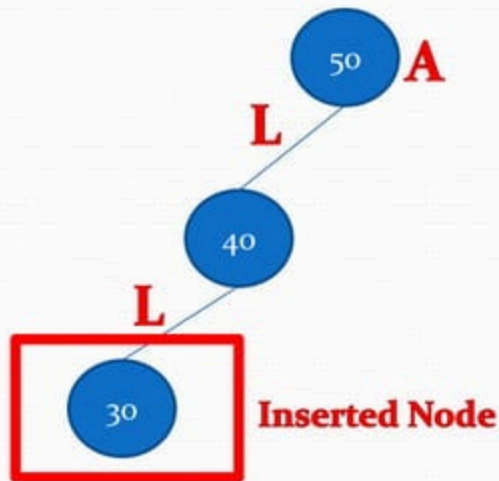
} Double Rotation

# SINGLE ROTATION

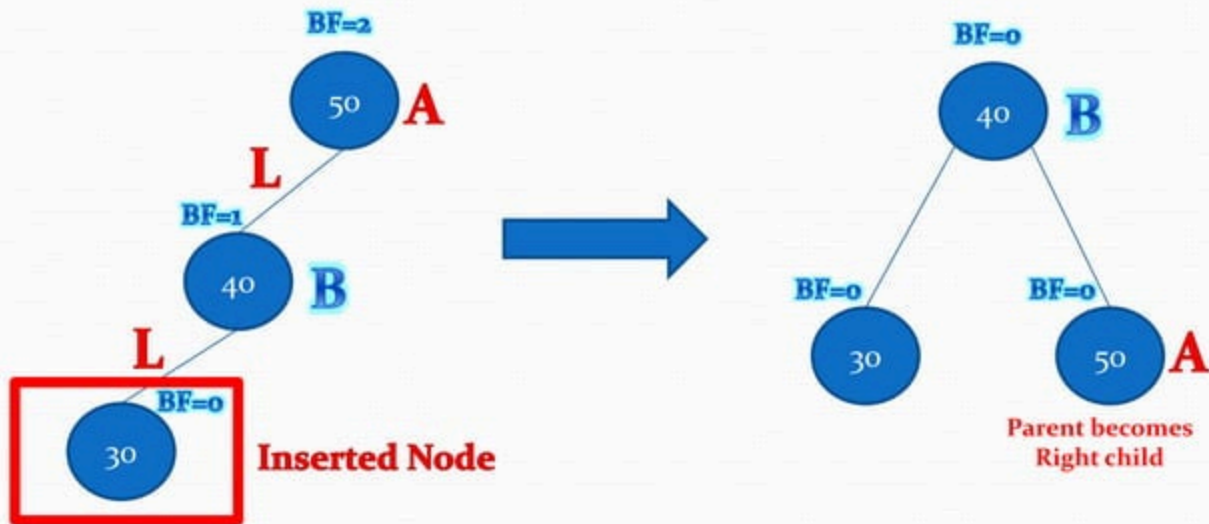
- Single rotation switches the roles of the parent and child while maintaining the search order.
- We rotate a node and its child, child becomes parent
- Parent becomes Right child in **LL Rotation**.
- Parent becomes Left child in **RR Rotation**.

# LEFT LEFT ROTATION

- Inserted node is in the left sub-tree of the left sub-tree of **A**.
- For LL Rotations identify **A** and **B**, where **B** is a Left child of **A** because insertion is on left side.
- Then make **A** as a child of **B**.

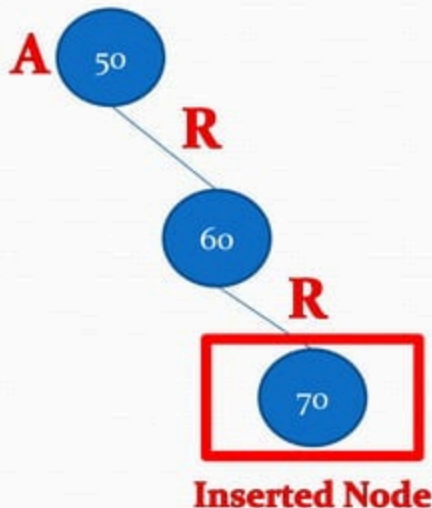


# LEFT LEFT ROTATION

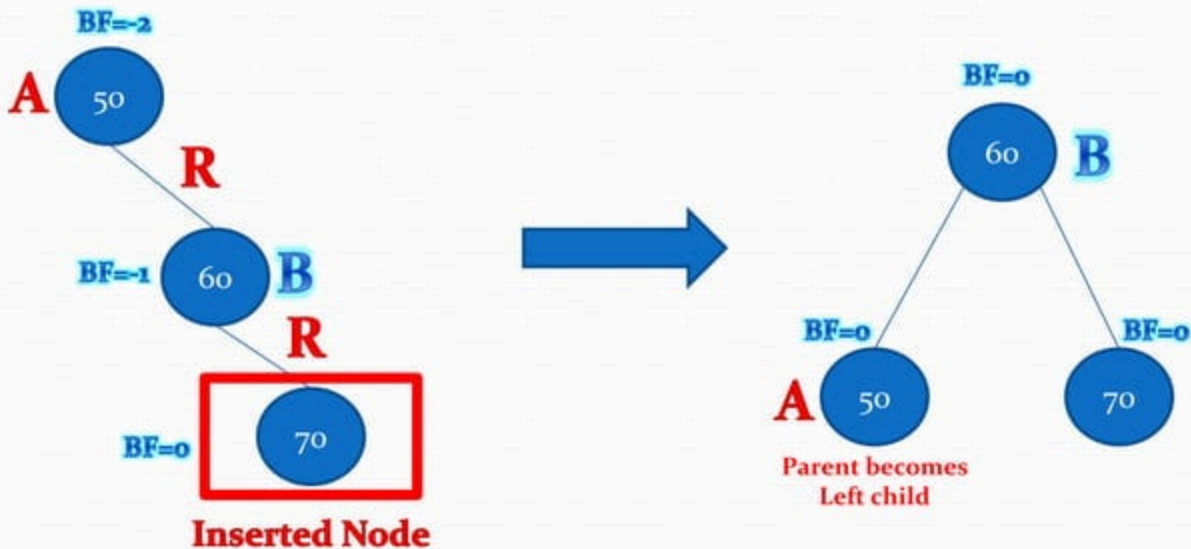


# RIGHT RIGHT ROTATION

- Inserted node is in the right sub-tree of the Right sub-tree of **A**.
- For RR Rotations identify **A** and **B**, where **B** is a Right child of **A** because insertion is on Right side.
- Then make **A** as a child of **B**.



# RIGHT RIGHT ROTATION



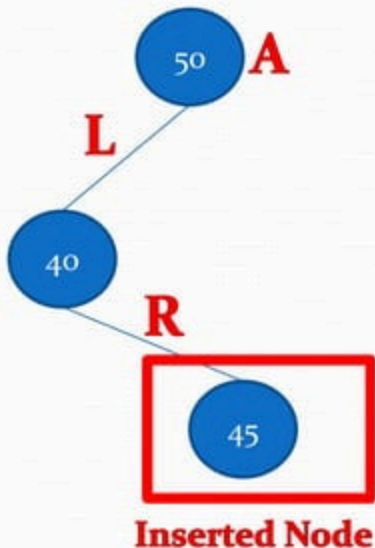
# DOUBLE ROTATION

- Single rotation does not fix the LR rotation and RL rotation.
- LR and RL rotations require a double rotation, involving three node.
- Double rotation is equivalent to a sequence of two single rotation.
  - 1<sup>st</sup> rotation on original tree
  - 2<sup>nd</sup> rotation on the new tree

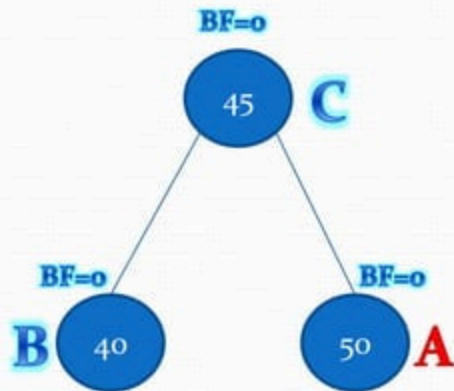
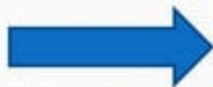
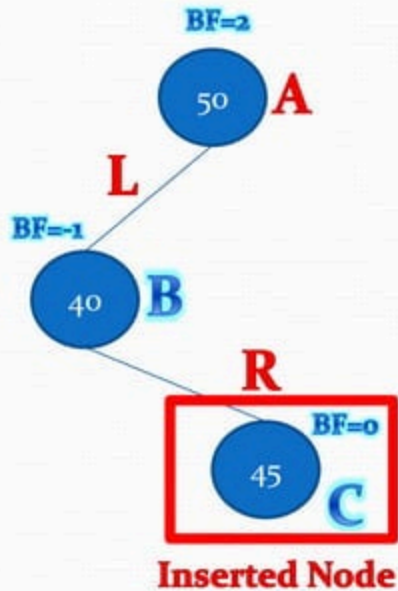


# LEFT RIGHT ROTATION

- Inserted node is in the right sub-tree of the left sub-tree of A.
- For LR Rotations identify **A**, **B** and **C**, where **B** is a Left child of **A** and **C** is right child of **B** because Inserted node is in the right sub-tree of the left sub-tree of **A**.
- Then make **A** and **B** as a child of **C**.

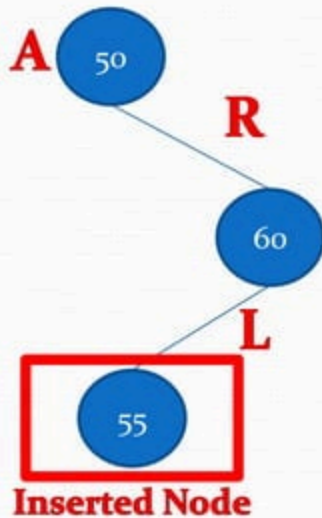


# LEFT RIGHT ROTATION

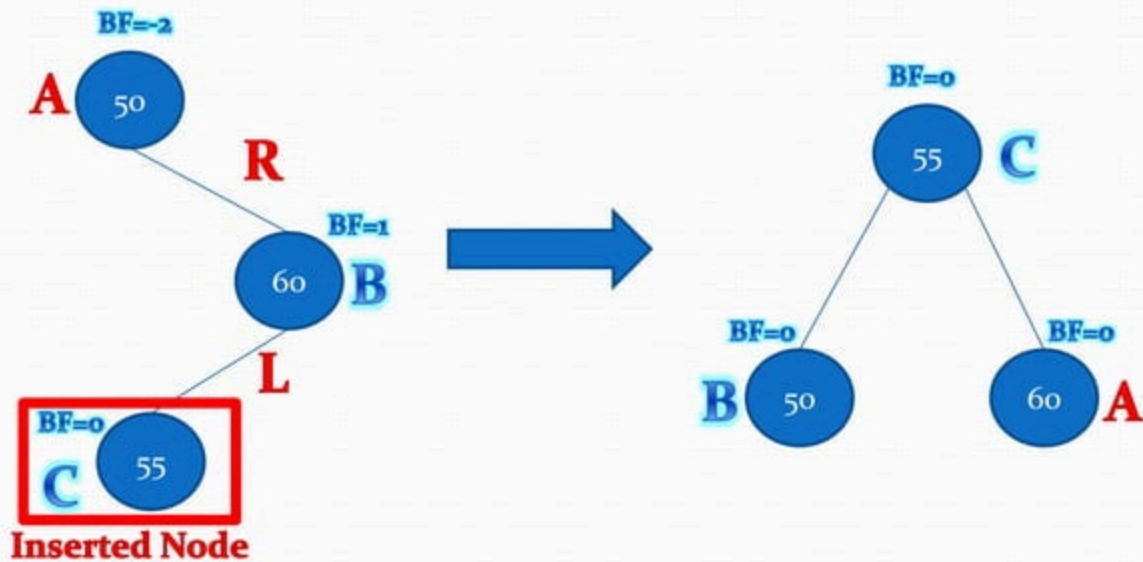


# RIGHT LEFT ROTATION

- Inserted node is in the left sub-tree of the Right sub-tree of A .
- For RL Rotations identify **A** **B** and **C**, where **B** is a Right child of **A** and **C** is Left child of **B** because Inserted node is in the left sub-tree of the Right sub-tree of **A**.
- Then make **A** and **B** as a child of **C**.



# RIGHT LEFT ROTATION



# INSERTION

- // If this node becomes unbalanced, then there are 4 cases

- // Left Left Case

- if (balance > 1 && key < node->left->key)
- return rightRotate(node);

- // Right Right Case

- if (balance < -1 && key > node->right->key)
- return leftRotate(node);

- // Left Right Case

- if (balance > 1 && key > node->left->key)
- {
- node->left = leftRotate(node->left);
- return rightRotate(node);
- }

- // Right Left Case

- if (balance < -1 && key < node->right->key)
- {
- node->right = rightRotate(node->right);
- return leftRotate(node);
- }

# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23

## GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23

BF=0





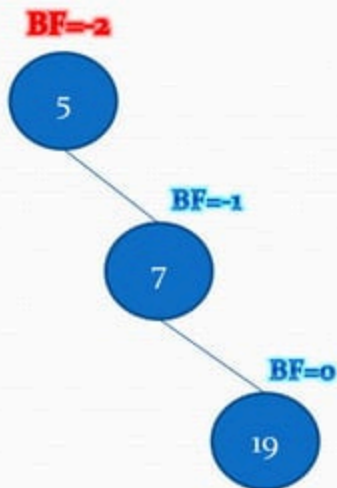
## GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



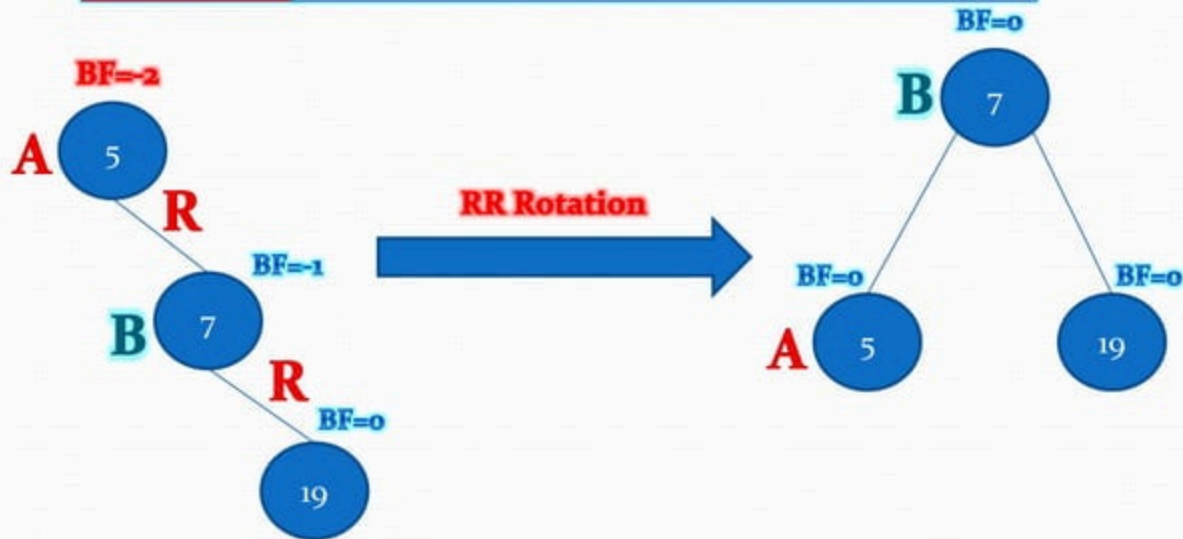
## GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



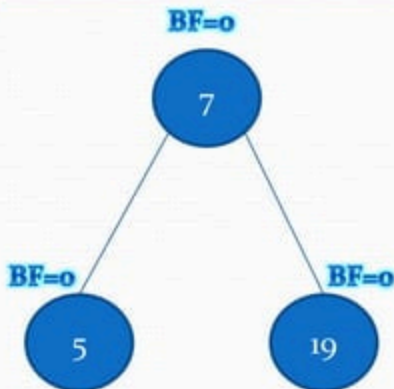
# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



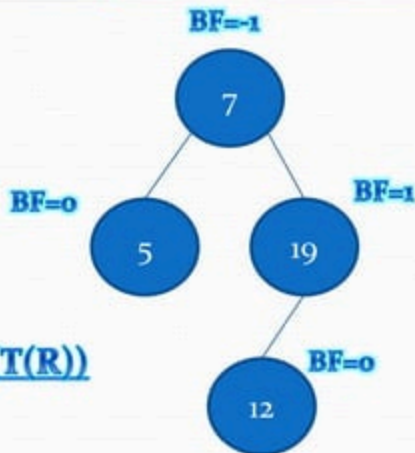
## GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



$$\underline{H(T(L)) = 1}$$

$$\underline{H(T(R)) = 2}$$

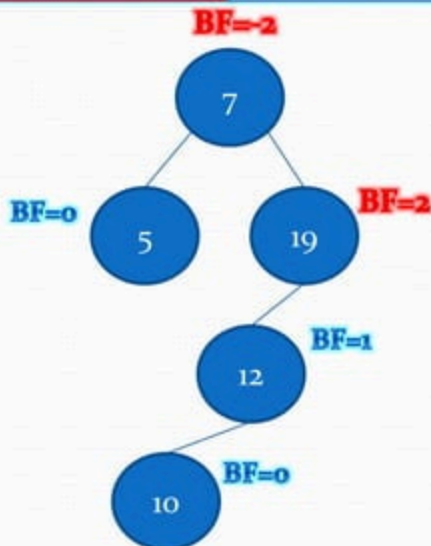
$$\underline{BF = H(T(L)) - H(T(R))}$$

$$\underline{BF = 1 - 2}$$

$$\underline{BF = -1}$$

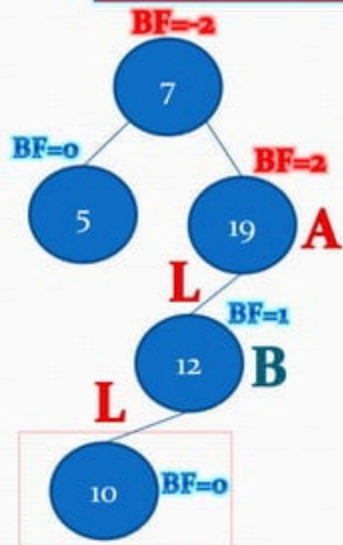
# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23

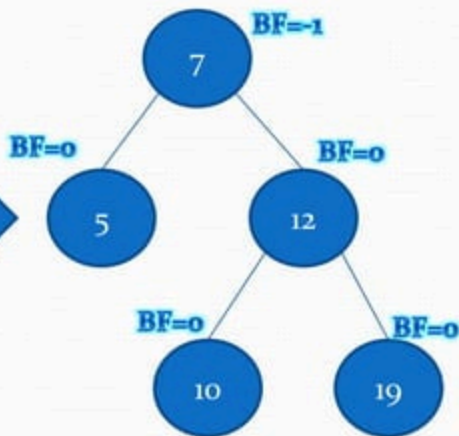
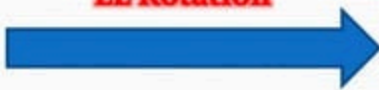


# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



LL Rotation



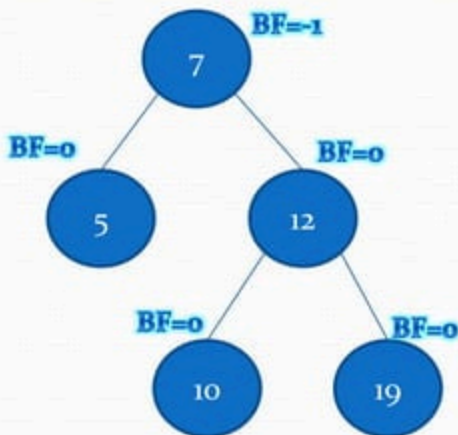
Inserted Node

Prepared By: Awais Ahmad



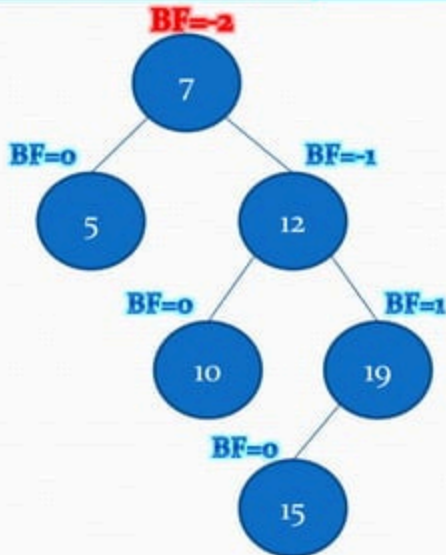
# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



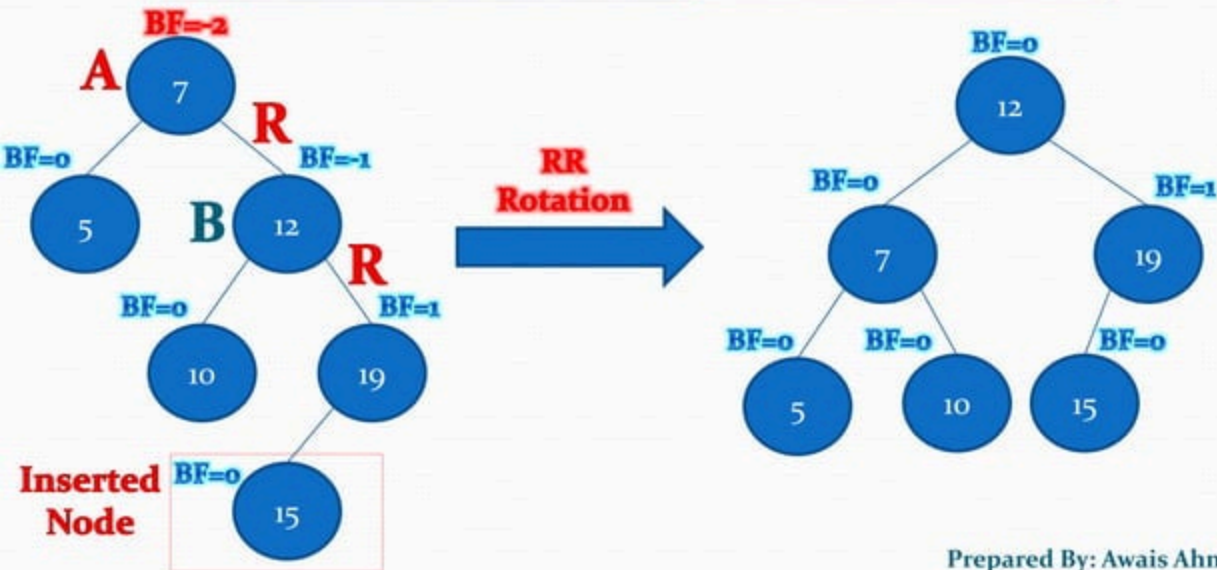
## GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



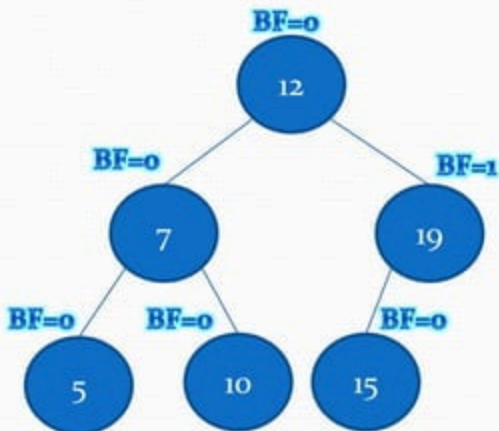
# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



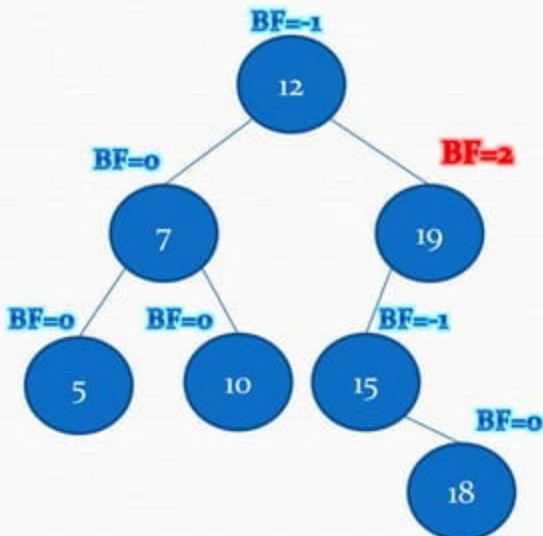
## GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



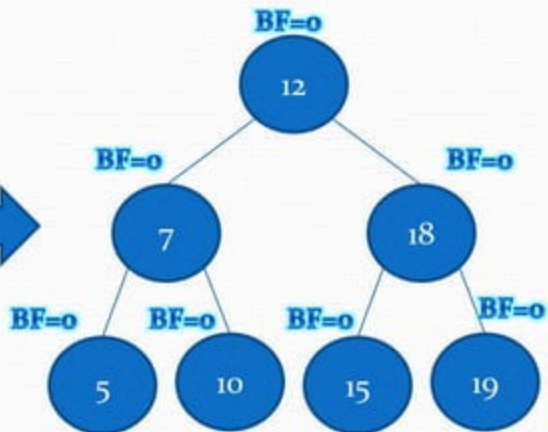
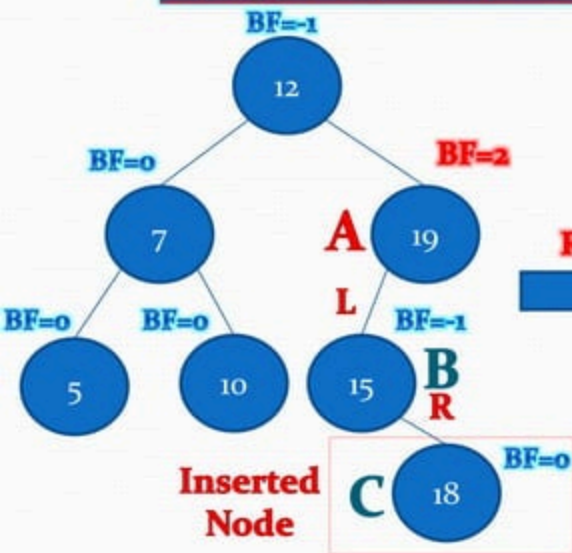
## GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



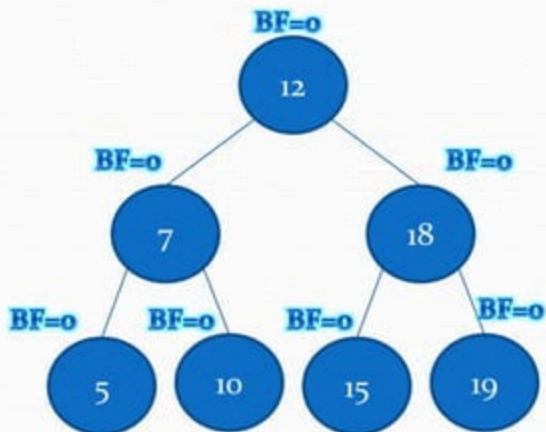
# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



# GENERATE AVL TREE FOR THE VALUES

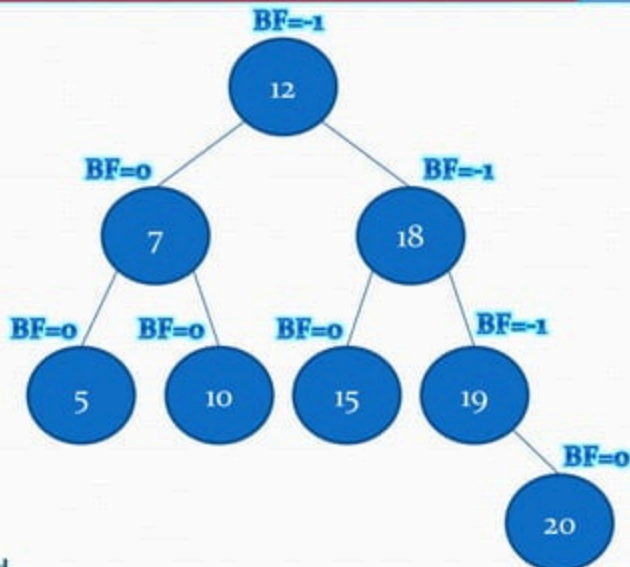
5, 7, 19, 12, 10, 15, 18, 20, 25, 23





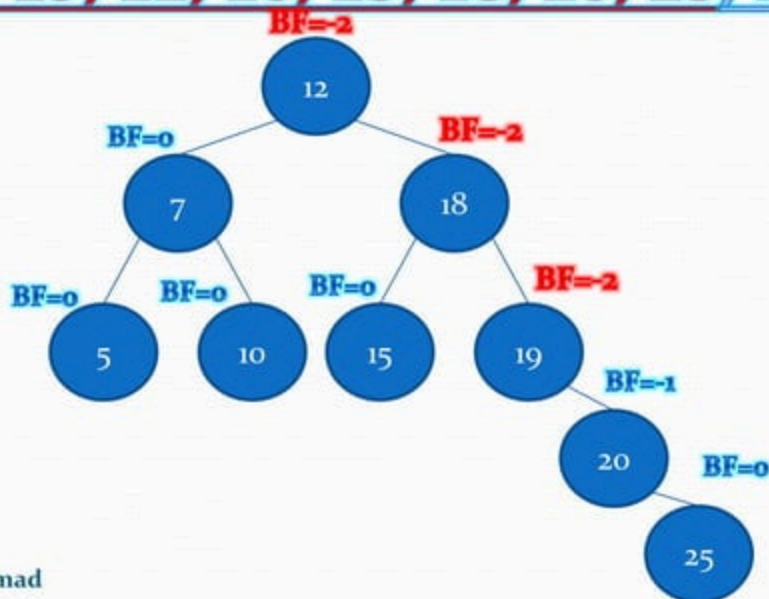
# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



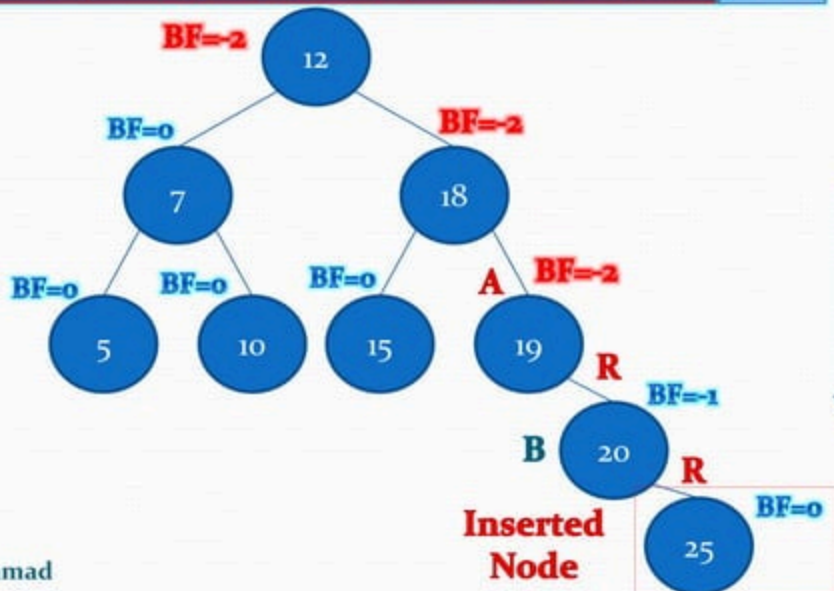
# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



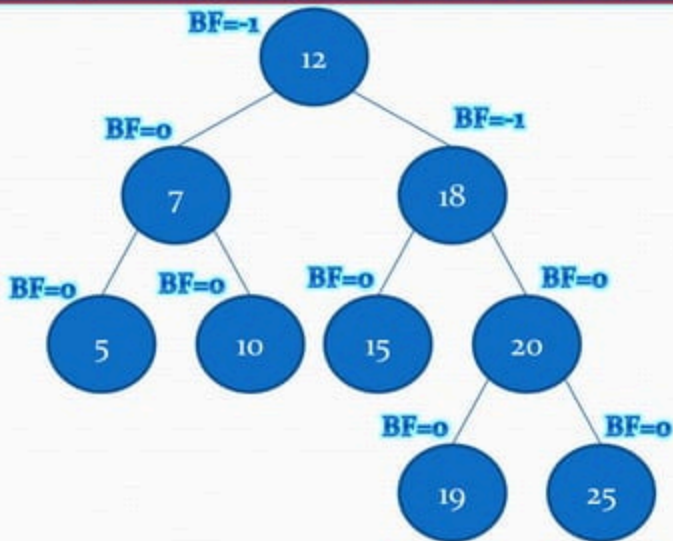
# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



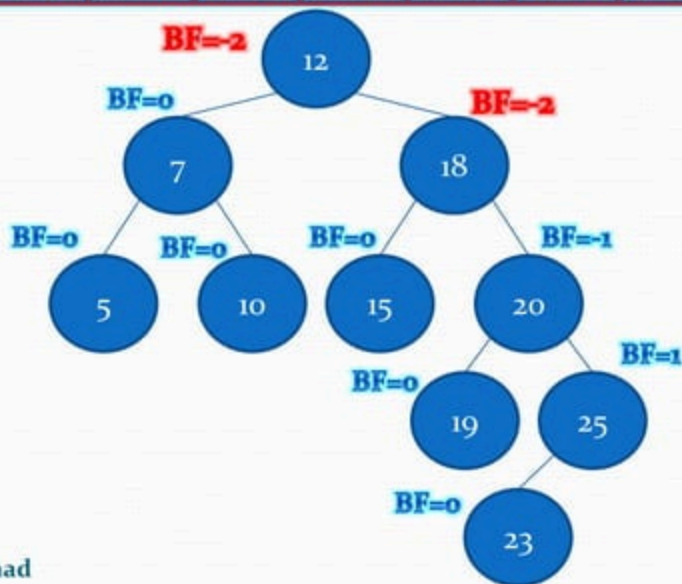
# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



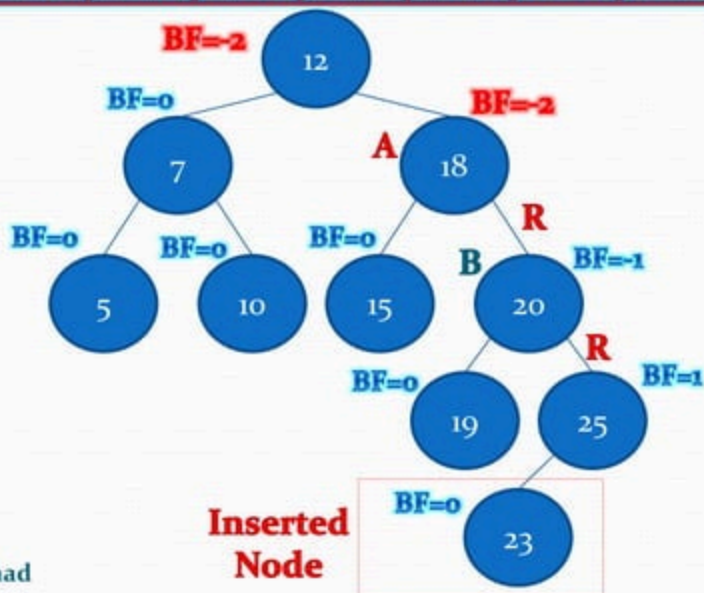
# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23



# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23





# GENERATE AVL TREE FOR THE VALUES

5, 7, 19, 12, 10, 15, 18, 20, 25, 23

